



Опрос про ИИ внутри разработки

[Войти](#)

ContentAI_Team

3 минуты назад

Как ИИ изменил разработку в Content AI: от эксперимента к новой реальности

8 мин

3

Блог компании Content AI, Управление разработкой*, Искусственный интеллект

Года полтора назад мы бы удивились, если бы услышали, что к лету 2026 разработчики начнут регулярно использовать ИИ-инструменты, тестировщики — автоматизировать с их помощью часть своей работы, а продакт-менеджеры — приносить на обсуждение первые (работающие!) прототипы, созданные с помощью ИИ. Но примерно так все и получилось.

Это третья статья из цикла про внедрение ИИ в процессы разработки Content AI. В первых двух мы рассказали, как [собрали ИИ-ревьюер кода за три дня](#) и как [научили ИИ писать код по нашим правилам](#). А теперь расскажем, что изменилось в работе всей команды разработки за полгода активного использования ИИ-инструментов. В этой статье мы не рассказываем о конкретных моделях или внутренней инфраструктуре. Нас интересует другое — как меняются роли людей и процессы разработки.

Как все начиналось

Точкой отсчета можно назвать лето 2025. По сути, мы тогда просто предложили команде: ребята, есть вот такой инструмент для разработки с ИИ-агентом, кто хочет, подключайтесь и пробуйте. В дополнение сделали отдельный чатик, кидали друг другу статьи по теме, обсуждали-спорили. Получился такой внутренний клуб по интересам.

Тогда ИИ-инструменты пробовала примерно пятая часть разработчиков, остальные посматривали скептически. И, честно говоря, не без оснований. В то время агенты на нашей кодовой базе работали так себе: код с историей, описания местами краткие, в

тестовом покрытии оставалось пространство для роста — в общем, для агента это было малоприспособно.

Где-то с января 2026 в агентах произошли качественные изменения — в промптах, обвязке и т.д. Они начали справляться с нашей легаси-базой, перестали заикаться и терять контекст, начали сами исследовать код и предлагать валидные решения. Для нас тогда стало открытием, что наша большая C++ кодовая база, которая специально не готовилась для работы с агентами, больше не является препятствием. Мы были уверены, что ИИ-инструменты — это про чистенькие проекты на модных фреймворках, а не про наше довольно сложное наследие. Если объяснять на простом примере — стало возможно скормить агенту описание бага и получить вменяемый алгоритм исправления и код. Агент научился из описания задачи предлагать план и его реализовывать.

Так у нас появились разработчики, которые стали использовать ИИ-агентов как основной инструмент при решении части задач даже в сложном технологическом стеке. При этом итоговые решения, как и прежде, проходили проверку и доработку со стороны команды. Примерно к концу января 2026 мы поняли, что этот подход работает и его надо масштабировать. Но решили это делать так же аккуратно, чтобы каждый разработчик попробовал инструмент и сам определил, где он полезен, с какими задачами в его репозитории справляется.

Сегодня ИИ-инструменты используют большинство разработчиков команды. При этом глубина использования сильно отличается: кто-то обращается к ним время от времени, а кто-то строит вокруг них практически весь рабочий процесс.

Как это происходило, хорошо видно по истории папки агента в нашем основном репозитории — там живут правила и скиллы. Первые правила появились еще в июне 2025: два файла на 16 строк в духе «вот так у нас принято писать C++ и QML». Дальше девять месяцев — лишь пара точечных правок, а в марте 2026 за три недели появились полноценный набор проектных правил (стиль, сборка, тесты) и первые скиллы. Скилл — это markdown-файл с пошаговой инструкцией, который агент сам подхватывает, распознав тип задачи: как собрать проект и прогнать тесты, как оформить Pull Request. В апреле добавились субагенты (планировщик, кодер, два ревьюера) и скилл-оркестратор, который гоняет их по циклу «план — код — ревью».

Скиллы оказались местом, куда стекается командный опыт: когда агент ошибается, мы правим не промпт в чате, а файл в репозитории, и через Pull Request это получают все. Показательно, что вскоре фиксы в скиллы начали присылать уже другие разработчики, не только авторы.

Каждый разработчик с ИИ стал немного тимлидом

Это, пожалуй, главная метафора года. Раньше схема была простая: разработчик сидит и пишет код по задаче. Теперь он формулирует требования, выступая как архитектор, советуется с агентом, придумывает несколько гипотез и просит агентов реализовать части этой задачи, то есть, по сути, ставит агенту задачу на разработку.

И вот тут произошла интересная вещь: уровень ответственности рядового разработчика вырос. Раньше со сложной задачей человек шел к тимлиду, а теперь может сразу обстучать ее в ИИ-редакторе. Но вместе с тем выросли и уровень того, что отдельный человек способен сделать сам, и уровень напряжения с принятием решений.

А вместе с этим произошел автоматический рост требований по скорости. Например, раньше было нормально сидеть над какой-то сложной задачей несколько дней. Теперь, если за два-три дня разработчик не показал хотя бы техническую записку или прототип, у команды возникает молчаливый вопрос: а что так долго, что говорит ИИ? Можно же было быстро набросать драфт с агентом. А если это не помогло, тогда уже зовем команду, грумим задачу, переходим в парное программирование и т.д. То есть с этим инструментом ожидания подскочили у всех разом. По сути, каждый становится мини-тимлидом для своих агентов.

А теперь про ускорение

Логичный вопрос: если разработка ускорилась, значит, и результата должно стать пропорционально больше? На самом деле все несколько сложнее.

Да, часть задач на уровне написания кода решается заметно быстрее. Но общий конвейер ускорился, по нашей оценке, процентов на 10%, а не в разы. Потому что написание кода — не главная часть работы. Главное — осмыслить требования, понять, какие именно изменения нужны, продумать архитектуру и краевые случаи. То есть архитектуру по-прежнему задает человек, ИИ-агент на себя не берет, он не осмысливает все связи внутри контекстов и не предлагает архитектуру целиком даже близко.

В итоге получается так: разработчик продумывает архитектуру решения, разбивает на модули, пишет подробный промт, и тогда агент справляется с прикладной реализацией. По сути, разработчик становится архитектором, а агент пишет тот самый пласт middleware, где и так понятно, что делать.

У такого ускорения есть и обратная сторона. Код, сгенерированный ИИ, требует внимательной проверки: разработчику важно понять логику решения, убедиться, что оно соответствует требованиям и не содержит скрытых проблем. В некоторых случаях такая проверка и доработка могут занять не меньше времени, чем если бы продумал и написал сам.

Где ускорение действительно бешеное, так это в задачах на ранний прототип. **Тут мы получали цифры роста скорости разработки в 5-10 раз.** Например, за месяц команда показывает рабочий MVP, который руками бы писали кратно дольше.

А вот на устоявшихся продуктах с большой кодовой базой ускорение скромнее, потому что аналитика, продумывание вариантов и проектирование архитектуры съедают основную долю времени, и агент тут почти не помощник.

Узкие места переехали к тимлидам и тестировщикам

Мы поняли, что когда разработка ускоряется, нагрузка просто перетекает дальше по конвейеру. Тимлидам приходится делать больше ревью и держать в голове больше контекстов одновременно, тестировать тоже нужно больше. Благодаря ИИ команда может быстро собрать первый технический MVP и использовать его для проверки гипотез еще до завершения детальной проработки требований и дизайна. Это позволяет раньше получать обратную связь и принимать более обоснованные продуктовые решения.

Из неожиданных эффектов также можно отметить изменение роли тестировщиков. Теперь специалист, даже без навыков программирования, делает запрос к агенту и получает код тестов. Правда, есть важный нюанс: агент хорошо пишет такой код, если человек сам придумал тестовые сценарии и четко их сформулировал. Поэтому роль тестировщика смещается с написания кода на проектирование сценариев. Пока агенты в целом плохо разбирают краевые сценарии и не справляются с задачей.

Агент в код-ревью и его неожиданный эффект

В прошлом году мы считали, что от агента в код-ревью толку мало. В этом — обнаружили, что комментарии стали по делу, качество выросло, то есть реальная польза налицо.

И тут случилось забавное. Мы сделали такое ревью опциональным, а разработчики быстро перестали смотреть Pull Request, пока его не прокомментировал агент. Логика простая: зачем делать двойную работу? Пусть сначала пройдет агент, а живой человек смотрит код уже после.

Но и тут есть подводные камни. Бывает, агент пишет 400 строк там, где нужно 40. И разработчик встает перед выбором: регенерить, написать руками или уговорить агента упростить логику. Есть целый класс задач, где промпт сочиняешь дольше, чем написал бы код сам. Понимание, когда стоит писать руками, а когда тратить силы на хороший промпт, — это отдельный навык, которому команда училась на ходу.

Почему страх разучиться писать код преувеличен

Часто поднимается вопрос, не опасно ли разработчику разучиться писать код руками? Тема реальная, но не новая. Она давно известна тимлидам, которые часто ловят себя на том, что пока ты менеджерил команду, навык написания кода руками просел. Но фокус в том, что вернуть навык и осваивать его с нуля — это разные вещи. То, что умел делать руками, восстанавливается очень быстро, когда ты возвращаешься к практике. Плюс здесь теперь есть агент, который подсказывает, куда смотреть и на что обращать внимание, поэтому порог входа обратно становится ниже.

Это можно сравнить с тем, как менялось потребление информации человеком с появлением смартфонов и поиска в интернете. То есть запоминать наизусть синтаксис и потеть над типовыми конструкциями, возможно, больше нет смысла — агент напишет их быстрее. А освободившееся внимание уходит на архитектуру и на пользовательский опыт. Чем больше рутины забирает агент, тем выше уровень задач, на котором работает разработчик.

Неожиданное применение ИИ-агента в менеджменте

Чего мы совсем не прогнозировали, так это изменения в менеджерской работе тех же тимлидов. Раньше письма, обратную связь, тексты обычно собирали в веб-интерфейсе чата, а теперь это переехало в агента разработчика. Просто потому что там удобно работать с контекстом.

Есть ощущение, что грамотным использованием такого инструмента можно усилить почти любую менеджерскую задачу. Потому что он делает аргументацию четче, предлагает альтернативные варианты, балансирует доводы.

Таким образом, меняются не только разработчики, следом за ними к новому подходу адаптируется менеджмент.

Но у этого тоже есть обратная сторона. Когда руководитель присылает в качестве обратной связи не три строчки, а пять страниц хорошо написанного текста, его надо прочитать и ухватить все смыслы. Аналогичные ситуации, когда разработчику прилетает большой pull request, который надо осознать, или менеджеру приходит объемное письмо, и проблема ровно та же.

Что мы вынесли из работы с агентами

Самое перспективное здесь то, что эти инструменты открыли нам задачи, которые раньше были недоступны с имеющимися ресурсами. Изменился и сам способ работы с требованиями, и появился неожиданный эффект: продакт-менеджеры сначала сами вайб-кодят прототип, а потом приходят с ним к команде разработки. Это переворачивает всю работу с требованиями: раньше путь шел сверху вниз, теперь продукт сам спускается до уровня прототипа и поднимается обратно с результатом.

Главную метрику для себя мы определили — это cycle time задач в разработке, по сути разработческая часть time-to-market. Цифры пока копим: фичи у нас большие, по два-три спринта, статистика размазывается, одна неудачная фича способна испортить картину. По ощущениям эффект позитивный, но подтвержденные выводы будут, когда наберется статистика.

Что ИИ меняет в требованиях к разработчикам

В начале этого года стало окончательно понятно: процесс неизбежен, все мы будем работать с этими инструментами.

Это уже отражается на найме. Мы все чаще обращаем внимание на готовность кандидата работать с современными ИИ-инструментами разработки. Не потому что это какой-то сакральный и повсеместно обязательный инструмент, а потому что неготовность воспринимается как сигнал, что человек не разобрался и не хочет разбираться с новым. Со студентами, кстати, таких проблем почти не наблюдается — их из этих инструментов не вытащишь. А вот среди опытных разработчиков встречается консерватизм, с которым уже бывает тяжело.

Так это революция или нет?

Для нас — totally да. Например, Anthropic публиковал перечень профессий, которые сильнее всего изменит ИИ, и разработчики там на первом месте. По нашим оценкам, на полную трансформацию у нас осталось 2-5 лет. Но это не абстрактные истории про то, что ИИ заменит человека, а видение будущего микса задач, где часть решается вместе с агентом, часть пишется руками (особенно там, где сложный код или самому просто быстрее).

Наш главный вывод за год с ИИ в разработке: ИИ-агент (пока) не заменяет архитектора, не осмысливает требования за разработчика и не может четко описать сценарии тестирования. Зато он забирает рутину и поднимает разработчика и тестировщика на уровень более сложных задач. При этом ответственность за архитектурные решения, проверку изменений и итоговое качество продукта по-прежнему остается за инженерами, а работа с ИИ-инструментами не отменяет требований информационной безопасности. Похоже, что это начало новой истории, где рассказывать о новых эффектах можно каждый месяц.

Расскажите, как ИИ-агенты меняют работу у вас, сверим опыт в комментариях.

А если интересен тот же путь глазами наших фронтендеров или продакт-менеджеров — тоже пишите, и мы подготовим продолжение.

Теги: разработка по, искусственный интеллект, ии, ии-агенты,
разработка программного обеспечения, управление разработкой

Хабы: Блог компании Content AI, Управление разработкой, Искусственный интеллект

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок

 **8K+**
Охват за 30 дней

Content AI

ВКонтакте Telegram Сайт

 **8K+** **139**
Охват за 30 дней Карма

@ContentAI_Team

Пользователь

Подписаться



Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



GlinkinIvan

23 часа назад

Хакер в магазине: изучаем поверхность атаки типичного супермаркета



9 мин



15K



+57



48



39



medicalofficer1111

23 часа назад

Пить collagen — это развод?



7 мин



9.6K



+37



40



24



arturdumchev

23 часа назад

Калвиаутра путает калвиши: один байт изменил всё



Средний



8 мин



11K

Тutorial



+37



8



7



PatientZero

3 часа назад

Теория мёртвой экономики



21 мин



7.8K

Перевод



+28



22



29



nastyakopi
22 часа назад

ИИ-роутер, open-source RAG-платформа, ClickHouse-as-a-service и другие апдейты в продуктах Selectel в июне

🕒 7 мин 👁 11K

Дайджест

📦 +24

🔖 7

💬 3



inetstar
21 час назад

Полный геном за \$200 и его анализ в домашних условиях: Just-DNA-Lite, ИИ и пересборка генома. Часть 3

🔗 Средний 🕒 29 мин 👁 10K

Тutorial

📦 +20

🔖 17

💬 4



SLY_G
20 часов назад

Сколько элементарных частиц существует на самом деле?

🔗 Простой 🕒 9 мин 👁 11K

Перевод

📦 +19

🔖 16

💬 12



xraizor
4 часа назад

Усталость от напарника-машины. Изнанка работы с ИИ-код-агентами

🔗 Средний 🕒 11 мин 👁 5.7K

Мнение

📦 +17

🔖 7

💬 9



AWE64
16 часов назад

Мне надоело писать один и тот же код. Поэтому я сделал Featuregen

 Средний  6 мин  13K

Кейс

 +16

 14

 4



inspired_xy

17 часов назад

Как мы съездили в Японию и запустили Пополаму — сервис, который знает кто кому должен

 Простой  4 мин  11K

Кейс

Из песочницы

 +16

 13

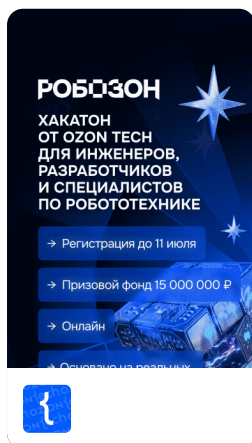
 4

Суперсилы и слабости IT-компаний: началось исследование работодателей 2026

Турбо

Показать еще

ИСТОРИИ



Унеси с Робозона свои миллионы



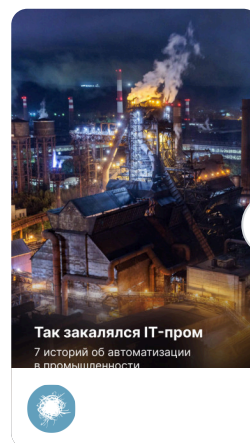
Пришёл, смастерил, рассказал — сезон DIY в разгаре



От лунного овоща до сверхострого чили



Из центра Млечного Пути

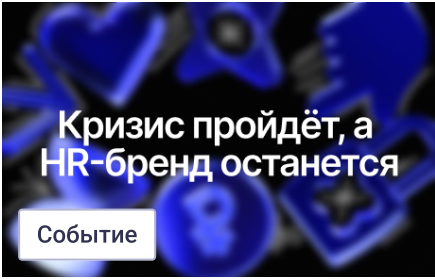


Только IT-пром, только хардкор



Промо

Отпуск? Обучение? Курсы? Есть скидки на все



Событие

Бесплатный вебинар о том, как строить HR-бренд в 2026 году



Опрос

Вы инженер в промышленности? Каким статьям вы доверяете?



🌐 Настройка языка

Техническая поддержка